JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Reference<T>

java.lang.Object
    java.lang.ref.Reference<T>

Type Parameters:

T - the type of the referent

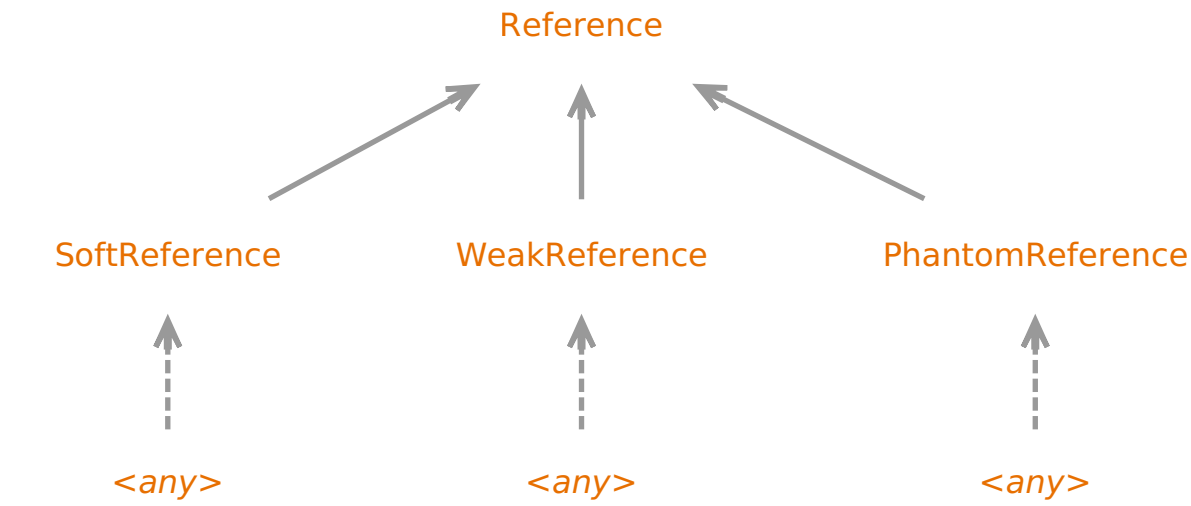
Direct Known Subclasses:

PhantomReference, SoftReference, WeakReference

```
public abstract sealed class Reference<T>
extends Object
permits PhantomReference<T>, SoftReference<T>, WeakReference<T> (not
exhaustive)
```

Abstract base class for reference objects. This class defines the operations common to all reference objects. Because reference objects are implemented in close cooperation with the garbage collector, this class may not be subclassed directly.

Sealed Class Hierarchy Graph:



Since:

1.2

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
void	clear()	

		Clears this reference object.
protected Object	**clone**()	Throws CloneNotSupportedException.
boolean	**enqueue**()	Clears this reference object, then attempts to add it to the queue with which it is registered, if any.
T	**get**()	Returns this reference object's referent.
boolean	**isEnqueued**()	**Deprecated.** This method was originally specified to test if a reference object has been cleared and enqueued but was never implemented to do this test.
static void	**reachabilityFence**(Object ref)	Ensures that the given object remains *strongly reachable*.
final boolean	**refersTo**(T obj)	Tests if the referent of this reference object is `obj`.

Methods declared in class Object

equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Details

get

```
public T get()
```

Returns this reference object's referent. If this reference object has been cleared, either by the program or by the garbage collector, then this method returns `null`.

API Note:

This method returns a strong reference to the referent. This may cause the garbage collector to treat it as strongly reachable until some later collection cycle. The `refersTo` method can be used to avoid such strengthening when testing whether some object is the referent of a reference object; that is, use `ref.refersTo(obj)` rather than `ref.get() == obj`.

Returns:

The object to which this reference refers, or `null` if this reference object has been cleared

See Also:

`refersTo(T)`

refersTo

```
public final boolean refersTo(T obj)
```

Tests if the referent of this reference object is `obj`. Using a `null` `obj` returns `true` if the reference object has been cleared.

Parameters:

`obj` - the object to compare with this reference object's referent

Returns:

`true` if `obj` is the referent of this reference object

Since:

16

clear

```
public void clear()
```

Clears this reference object. Invoking this method does not enqueue this object, and the garbage collector will not clear or enqueue this object.

When the garbage collector or the `enqueue()` method clear references they do so directly, without invoking this method.

API Note:

There is a potential race condition with the garbage collector. When this method is called, the garbage collector may already be in the process of (or already completed) clearing and/or enqueueing this reference. Avoid this race by ensuring the referent remains strongly reachable until after the call to clear(), using `reachabilityFence(Object)` if necessary.

isEnqueued

```
@Deprecated(since="16")
public boolean isEnqueued()
```

Deprecated.
This method was originally specified to test if a reference object has been cleared and enqueued but was never implemented to do this test. This method could be misused due to the inherent race condition or without an associated `ReferenceQueue`. An application relying on this method to release critical resources could cause serious performance issue. An application should use `ReferenceQueue` to reliably determine what reference objects that have been enqueued or `refersTo(null)` to determine if this reference object has been cleared.

Tests if this reference object is in its associated queue, if any. This method returns `true` only if all of the following conditions are met:

- this reference object was registered with a queue when it was created; and
- the garbage collector has added this reference object to the queue or `enqueue()` is called; and
- this reference object is not yet removed from the queue.

Otherwise, this method returns `false`. This method may return `false` if this reference object has been cleared but not enqueued due to the race condition.

Returns:

`true` if and only if this reference object is in its associated queue (if any).

enqueue

```
public boolean enqueue()
```

Clears this reference object, then attempts to add it to the queue with which it is registered, if any.

If this reference is registered with a queue but not yet enqueued, the reference is added to the queue; this method is ***successful*** and returns true. If this reference is not registered with a queue, or was already enqueued (by the garbage collector, or a previous call to enqueue), this method is ***unsuccessful*** and returns false.

Memory consistency effects: Actions in a thread prior to a ***successful*** call to enqueue *happen-before* the reference is removed from the queue by `ReferenceQueue.poll()` or `ReferenceQueue.remove(long)`. ***Unsuccessful*** calls to enqueue have no specified memory consistency effects.

When this method clears references it does so directly, without invoking the `clear()` method. When the garbage collector clears and enqueues references it does so directly, without invoking the `clear()` method or this method.

API Note:

Use of this method allows the registered queue's `ReferenceQueue.poll()` and `ReferenceQueue.remove(long)` methods to return this reference even though the referent may still be strongly reachable.

Returns:

`true` if this reference object was successfully enqueued; `false` if it was already enqueued or if it was not registered with a queue when it was created

clone

```
protected Object clone()
                throws CloneNotSupportedException
```

Throws `CloneNotSupportedException`. A `Reference` cannot be meaningfully cloned. Construct a new `Reference` instead.

Overrides:

`clone` in class `Object`

Returns:

never returns normally

Throws:

`CloneNotSupportedException` - always

See Also:

`Cloneable`

reachabilityFence

```
public static void reachabilityFence(Object ref)
```

Ensures that the given object remains *strongly reachable*. This reachability is assured regardless of any optimizing transformations the virtual machine may perform that might otherwise allow the object to become unreachable (see JLS §12.6.1). Thus, the given object is not reclaimable by garbage collection at least until after the invocation of this method. References to the given object will not be cleared (or enqueued, if applicable) by the garbage collector until after invocation of this method. Invocation of this method does not itself initiate reference processing, garbage collection, or finalization.

This method establishes an ordering for *strong reachability* with respect to garbage collection. It controls relations that are otherwise only implicit in a

program -- the reachability conditions triggering garbage collection. This method is applicable only when reclamation may have visible effects, such as for objects that use finalizers or `Cleaner`, or code that performs reference processing.

Memory consistency effects: Actions in a thread prior to calling `reachabilityFence(x)` *happen-before* the garbage collector clears any reference to `x`.

API Note:

Reference processing or finalization can occur after an object becomes unreachable. An object can become unreachable when the virtual machine detects that there is no further need for the object (other than for running a finalizer). In the course of optimization, the virtual machine can reorder operations of an object's methods such that the object becomes unneeded earlier than might naively be expected — including while a method of the object is still running. For instance, the VM can move the loading of *values* from the object's fields to occur earlier. The object itself is then no longer needed and becomes unreachable, and the method can continue running using the obtained values. This may have surprising and undesirable effects when using a Cleaner or finalizer for cleanup: there is a race between the program thread running the method, and the cleanup thread running the Cleaner or finalizer. The cleanup thread could free a resource, followed by the program thread (still running the method) attempting to access the now-already-freed resource. Use of `reachabilityFence` can prevent this race by ensuring that the object remains strongly reachable.

The following is an example in which the bookkeeping associated with a class is managed through array indices. Here, method `action` uses a `reachabilityFence` to ensure that the `Resource` object is not reclaimed before bookkeeping on an associated `ExternalResource` has been performed; specifically, to ensure that the array slot holding the `ExternalResource` is not nulled out in method `Object.finalize()`, which may otherwise run concurrently.

```
class Resource {
  private static ExternalResource[] externalResourceArray = ...

  int myIndex;
  Resource(...) {
    this.myIndex = ...
    externalResourceArray[myIndex] = ...;
    ...
  }
  protected void finalize() {
```

```
    externalResourceArray[this.myIndex] = null;
    ...
  }
  public void action() {
    try {
      // ...
      int i = this.myIndex; // last use of 'this' Resource in action()
      Resource.update(externalResourceArray[i]);
    } finally {
      Reference.reachabilityFence(this);
    }
  }
  private static void update(ExternalResource ext) {
    ext.status = ...;
  }
}
```

The invocation of `reachabilityFence` is placed *after* the call to `update`, to ensure that the array slot is not nulled out by `Object.finalize()` before the update, even if the call to `action` was the last use of this object. This might be the case if, for example, a usage in a user program had the form `new Resource().action();` which retains no other reference to this `Resource`. The `reachabilityFence` call is placed in a `finally` block to ensure that it is invoked across all paths in the method. A more complex method might need further precautions to ensure that `reachabilityFence` is encountered along all code paths.

Method `reachabilityFence` is not required in constructions that themselves ensure reachability. For example, because objects that are locked cannot, in general, be reclaimed, it would suffice if all accesses of the object, in all methods of class `Resource` (including `finalize`) were enclosed in `synchronized (this)` blocks. (Further, such blocks must not include infinite loops, or themselves be unreachable, which fall into the corner case exceptions to the "in general" disclaimer.) However, method `reachabilityFence` remains a better option in cases where synchronization is not as efficient, desirable, or possible; for example because it would encounter deadlock.

Parameters:

`ref` - the reference to the object to keep strongly reachable. If `null`, this method has no effect.

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.